

DevPilot AI

Agentic Engineering & Delivery Command Center

Implementation Plan | Architecture | Step-by-step Execution Guide

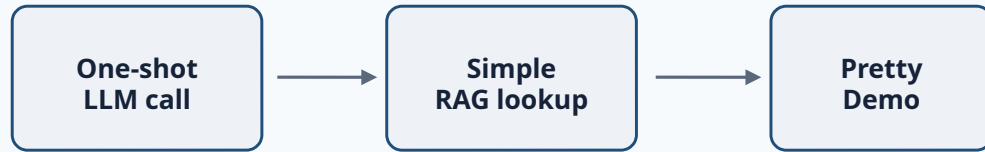


Built to stand out as a public portfolio project: .NET + Azure + RAG + LangGraph + DevOps + Security + Observability

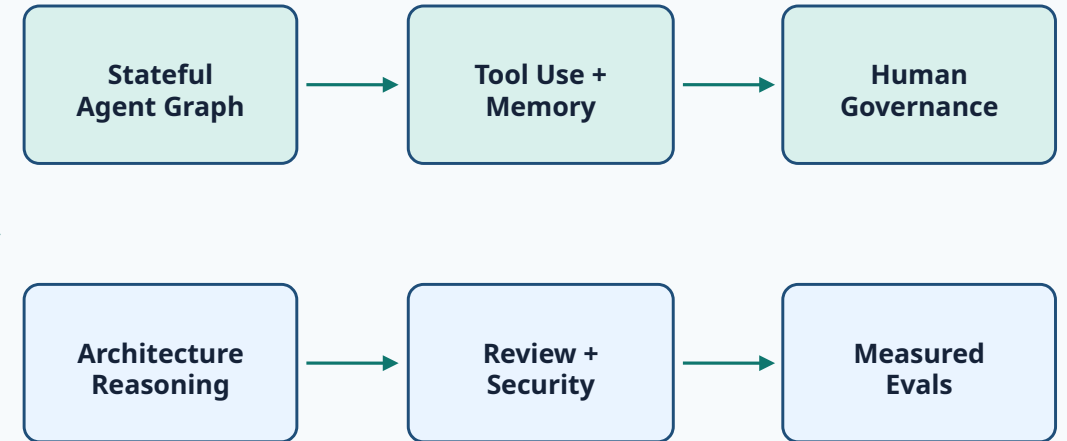
Why DevPilot AI stands out

It demonstrates end-to-end engineering judgment, not just prompt output.

Most AI portfolio projects stop here



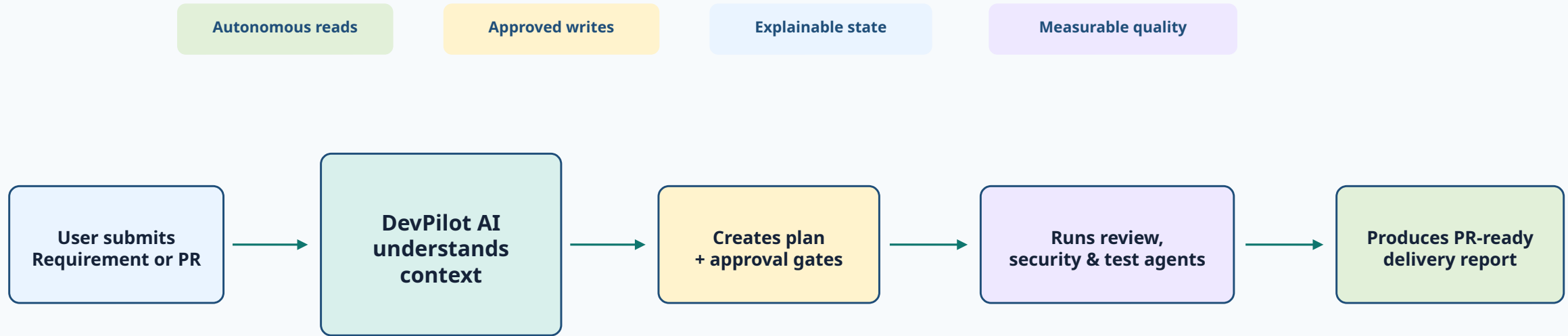
DevPilot AI goes here



Career signal: Technical Lead + Solution Architect + AI Engineer + DevOps story in one coherent build.

Product vision

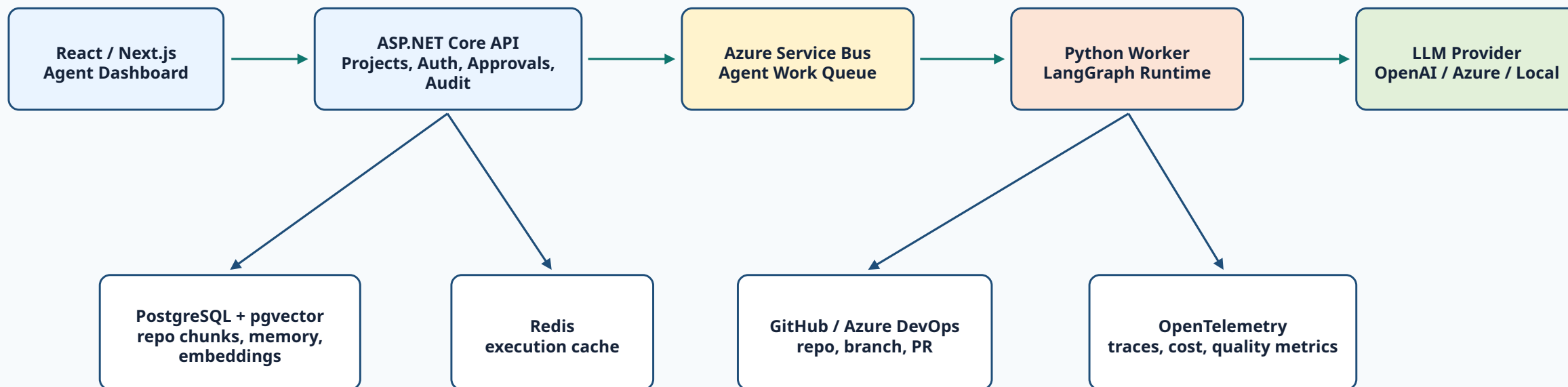
An AI Engineering Lead that converts requirements into delivery-ready engineering output.



One-liner: DevPilot AI helps engineering leads convert software requirements into architecture, implementation plans, code intelligence, review findings, security checks, testing strategy, and delivery-ready pull requests through a stateful multi-agent workflow.

Target architecture

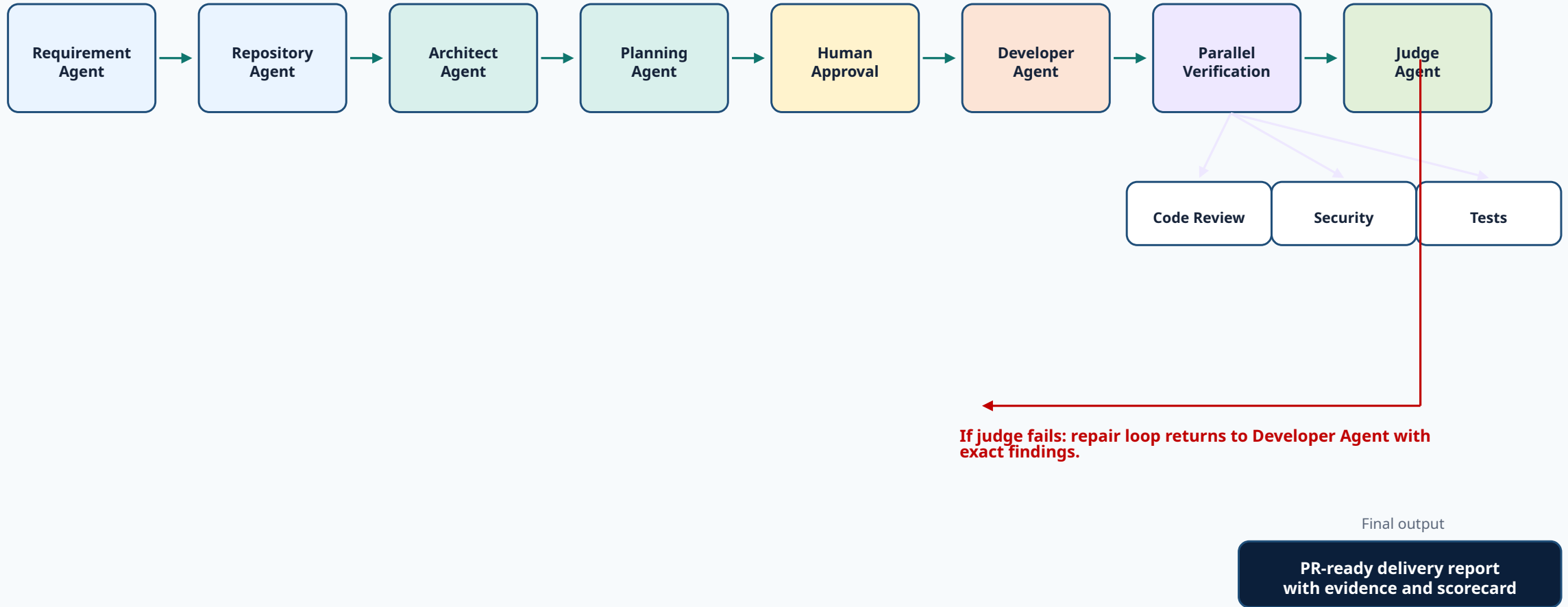
Split product API and agent runtime so the project feels production-grade.



Architecture message: enterprise .NET product shell + specialized Python agent brain + Azure-native deployment path.

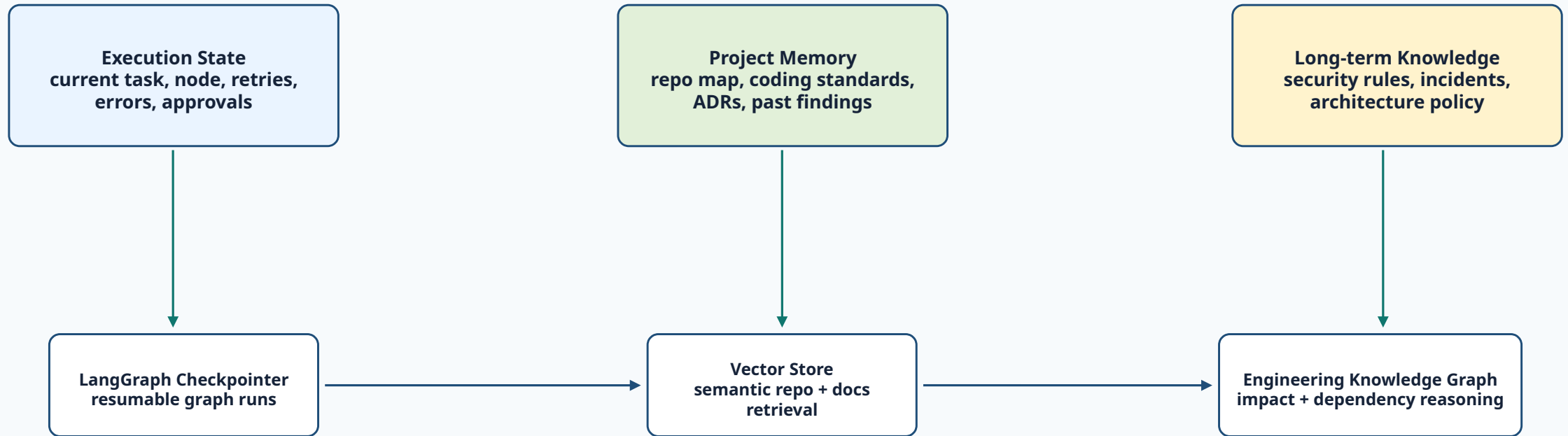
Multi-agent workflow

The orchestrator routes specialized agents through a controlled delivery lifecycle.



State, memory, and knowledge

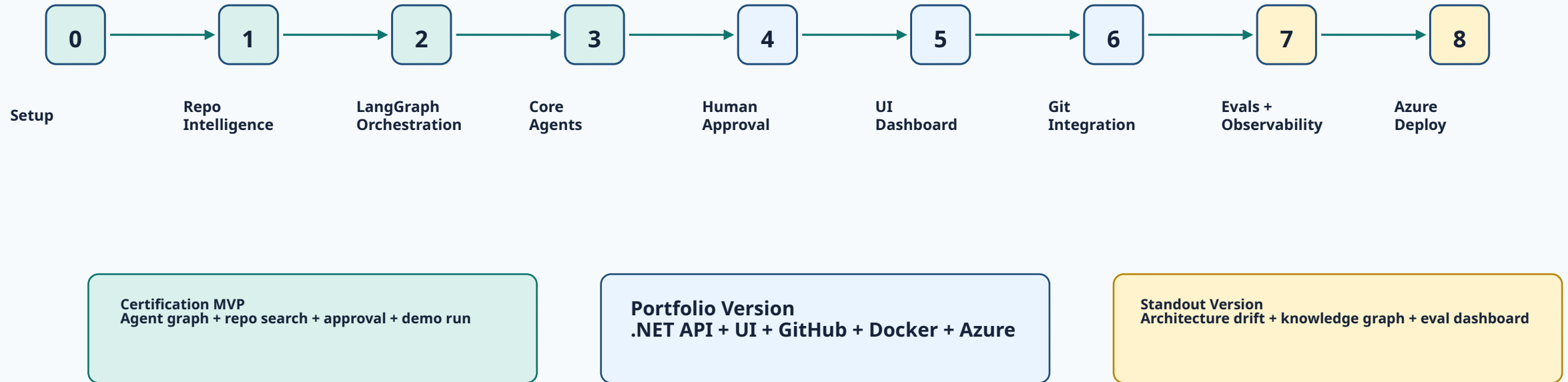
Make the hard part visible: what the agent remembers and where it lives.



This is the difference between chat history and engineering-grade application state.

Implementation roadmap

Build small, complete, and demonstrable first - then deepen the portfolio version.

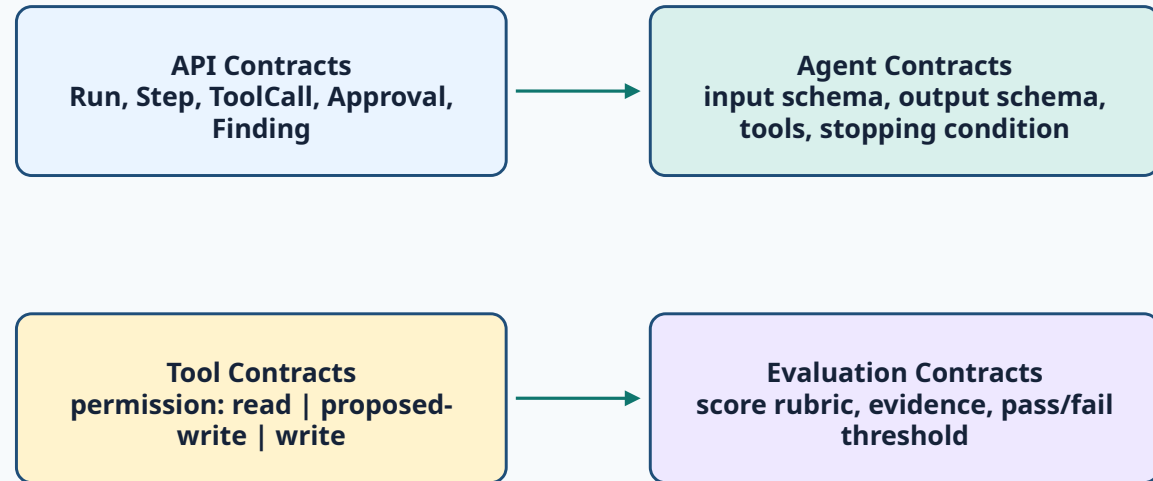


Repository structure and contracts

A clean mono-repo keeps the implementation explainable.

devpilot-ai/

```
src/  
  apidotnet/  
    Api | Application | Domain | Infrastructure  
  agentpython/  
    graphs | agents | tools | memory | evals  
  web/  
    app | components | agent-run-ui  
infra/  
  docker | azure | github-actions  
docs/  
  architecture | decisions | demo  
samples/  
  dotnet-commerce-api/
```



Design rule: every agent and tool must be testable independently before it enters the LangGraph workflow.

Human-in-the-loop and guardrails

The system earns trust by controlling what can happen autonomously.



Governance model

Every tool call is classified, logged, and tied to a run step. Every write request carries risk level, evidence, and explicit human approval.

Observability and evaluation

Move the story from “AI generated output” to “AI workflow measured quality.”

94%

Architecture
Compliance

97%

Tool Call
Success

11%

Human
Intervention

72s

Median
Run Time

\$0.18

Estimated
Cost / Run

Evaluation rubric: requirement completeness, architecture compliance, security risk, test coverage, code quality, hallucination rate, latency, cost, retry count, and approval rate.

LangSmith / traces

OpenTelemetry

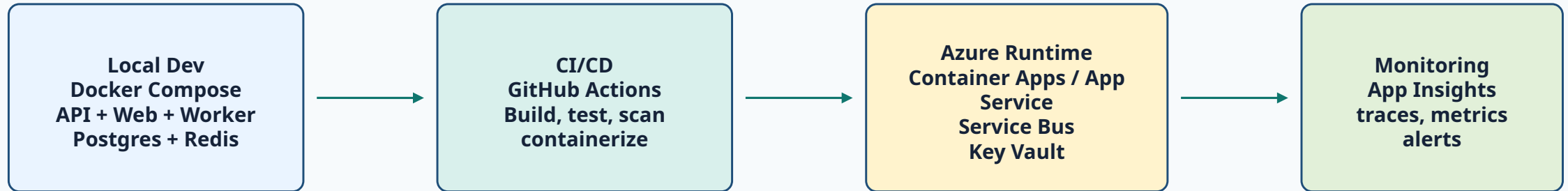
Application Insights

Custom eval suite

Run scorecard

Execution and Azure deployment path

Start local with Docker Compose; deploy the same boundaries to Azure.



Execution demo task: "Add Redis distributed caching to Product API using cache-aside strategy with safe fallback."

Final demo and publishing checklist

A strong README and 5-minute demo will make the project discoverable and credible.

5-minute demo flow

1. Introduce problem and product vision
2. Show architecture and agent workflow
3. Submit sample requirement
4. Watch agent trace and repository discovery
5. Approve plan before write action

Portfolio polish

6. Show review/security/test agents
7. Demonstrate one failure recovery path
8. Show final scorecard and delivery report
9. Publish GitHub README with diagrams
10. Add roadmap: drift detection + knowledge graph

Ship the MVP first. Then add the standout features.